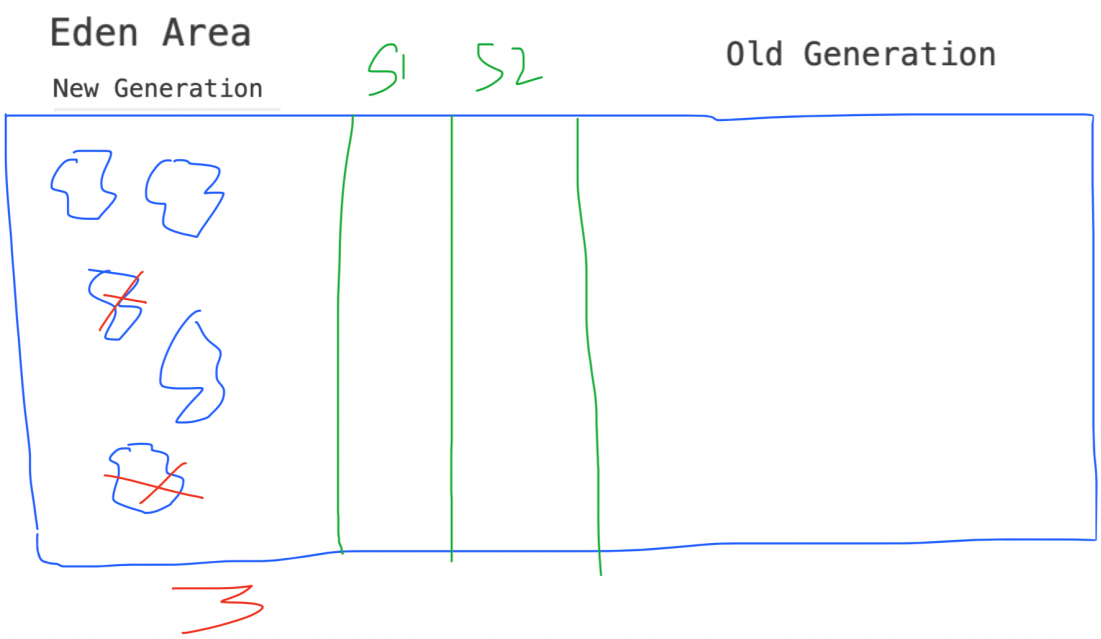
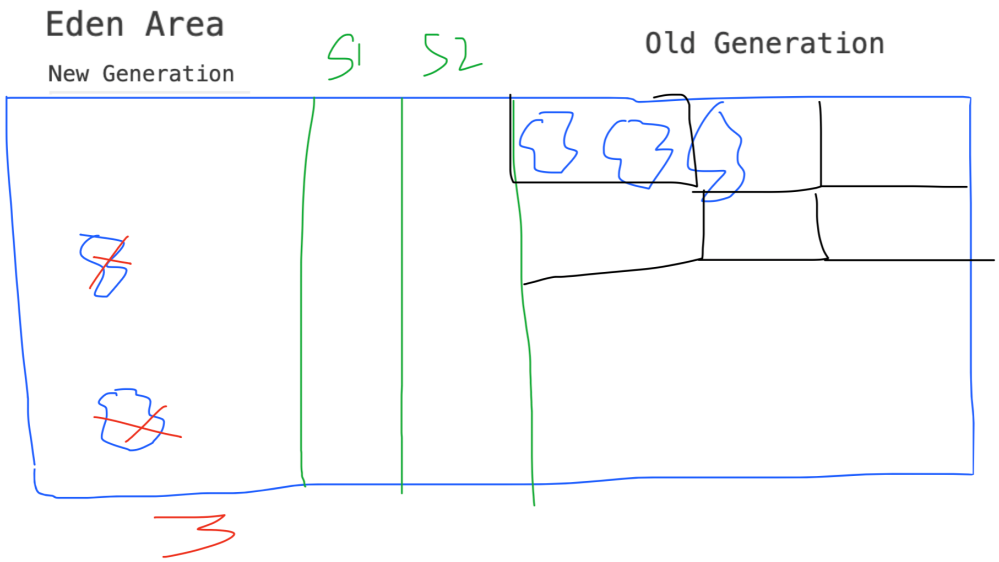





```
function adder(base) {
    return function(no) {
        return base + no;
    }
}

var fiveAdder = adder(5);
```

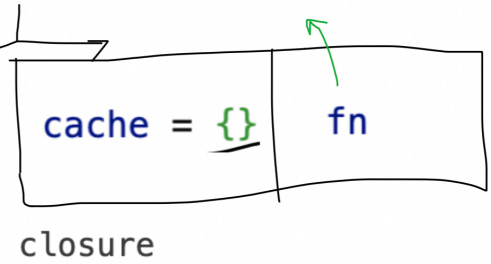
closure

5

base

```
> fiveAdder.prototype
<· ▼ {} i
    ▼ constructor: ƒ (no)
        arguments: null
        caller: null
        length: 1
        name: ""
      ▶ prototype: {}
        [[FunctionLocation]]: VM15383:2
      ▶ [[Prototype]]: ƒ ()
      ▼ [[Scopes]]: Scopes[2]
        ▶ 0: Closure (adder) {base: 5}
        ▶ 1: Global {window: Window, self: Window, document: document, name: 'Samsung', location: Location, …}
    ▶ [[Prototype]]: Object
```

```
function fibanocci(no) {
    return ((no == 0) || (no == 1)) ? no : fibanocci(no - 1) + fibanocci(no - 2);
}
```

```
var memFib = memoize(fibanocci);
```

```
function memoize(fn) {
    var cache = {};
    return function(args) {
        if(!cache[args]) {
            cache[args] = fn(args);
        }
        return cache[args];
    }
}
```